

Exploring the Feasibility and Performance of Distributed LLM Serving on Consumer-grade GPUs

Lewei Jin, Yang Liu, Yongqi Chen, Xiaoqiang Guo, Kui Zhang, Yi Gao, and Wei Dong

College of Computer Science and Technology, Zhejiang University, China

Email: {jinlewei, quakso, yqcchen, guoxq, kuizhang, gaoyi, dongw}@zju.edu.cn

Abstract—The advances in large language models come with a challenge: the high cost of model serving. Consumer-grade GPUs emerge as a cost-efficient alternative. However, there remains a gap in the systematic study of their feasibility and performance.

In this work, we present the first comprehensive study of LLM serving on consumer-grade GPUs. We characterize serving performance using six representative benchmarks and uncover four key findings that are often masked in traditional datacenter-centric studies: (1) Consumer-grade GPUs offer low latency inference but suffer from low concurrency due to limited memory. (2) Tensor parallelism does not always provide benefits due to PCIe bandwidth constraints. Switching to pipeline parallelism can mitigate this issue. (3) Scaling pipeline parallelism yields superlinear throughput gains, but these gains diminish under lower network bandwidth. (4) State-of-the-art serving systems suffer from two typical limitations: (i) prefill requests blocking decode requests, and (ii) underutilized memory in heterogeneous hardware settings. To address these limitations, we propose two simple yet effective techniques. Experiments show that our approach reduces Time to First Token (TTFT) by up to 37.82%, Time Per Output Token (TPOT) by up to 18.72%, and improves overall throughput by up to 21.86% compared to the baselines. Furthermore, it achieves up to 11.2% higher throughput in heterogeneous settings.

I. INTRODUCTION

Large language models are reshaping existing Internet services. Yet, these advances come with a challenge: the high cost of model serving. To mitigate these costs, consumer-grade GPUs have emerged as an alternative. Taking the RTX 4090 as an example, its computational performance of 330 TFLOPs is comparable to that of the A100 at 312 TFLOPs, yet it costs only one-tenth as much to procure. It is reported that the PC and AIB GPU market shipped over 101 million units in Q4 2021 alone [1]. This presents an opportunity for more cost-efficient LLM serving by leveraging these GPUs.

This insight has inspired many LLM serving systems on consumer-grade GPUs [2]–[8]. Some of them introduce fault-tolerant inference for devices that can disconnect abruptly [4]. Others aim to reduce memory overhead; for example, vLLM [5] introduces PagedAttention, which allocates a paged KV cache. Others target heterogeneous consumer GPU environments. Helix [6] proposes a max-flow abstraction of clusters

and finds the optimal deployment to maximize the throughput of LLM serving in heterogeneous GPU environments.

However, the feasibility of serving LLMs on consumer-grade GPUs remains unclear. Recent measurement studies have evaluated aspects of LLM serving [9]–[12]. They typically focus on professional-grade GPUs [9] or rely on auxiliary hardware, such as CPU offloading [12]. Therefore, their insights may not be directly applicable, since consumer-grade GPUs have unique characteristics, such as limited memory and constrained networking capabilities in non-RDMA or non-NVLink environments.

In this paper, we present the *first* measurement study on leveraging consumer-grade GPUs for serving LLMs. This is non-trivial, as many configuration factors affect serving performance. To address this, we design six representative benchmarks at different levels, including model size, prompt length, degree of parallelism, network conditions. We evaluate the performance of consumer-grade GPUs across these benchmarks and uncover the following key findings.

Finding #1. Consumer-grade GPUs offer high compute capability, enabling low-latency inference for large language models, yet their limited memory capacity restricts request concurrency. For example, on four RTX 4090 GPUs running vLLM-V1, Qwen3-32B achieves a per-token latency of approximately 0.2 seconds. However, model weights alone consume 70.6% of the available memory, leaving little space for the KV cache needed during inference. The concurrent requests is only 26 at a context length of about 900 tokens, which is common in chat applications. In contrast, the smaller Qwen3-8B model supports up to 168 concurrent requests on the same hardware, while maintaining a similar latency. This shows a trade-off between model size and system throughput. Deploying smaller models or reducing context length can effectively improve concurrency.

Finding #2. Tensor parallelism does not always benefit LLM serving on consumer-grade GPUs due to limited inter-GPU bandwidth, which introduces a communication bottleneck. Hybrid parallelism mitigate this issue. Unintuitively, we observe that as prompt length increases, 4-way tensor parallelism results in worse time to first token (TTFT) than 4-way pipeline parallelism. For example, tensor parallelism is 2.3 times slower at a prompt length of 4K tokens. A similar trend also occurs with large decode batch (e.g., size is 64). It is because tensor parallelism requires frequent all-to-

This work is supported by the Major Program of Zhejiang NSF Grant No. LD26F020002, the National Natural Science Foundation of China under Grant No. 92582114, No. 625B2167, and No. 62272407. Wei Dong is the corresponding author.

all communication across GPUs, which becomes a bottleneck under PCIe’s limited bandwidth. Adopting a hybrid pipeline strategy helps to mitigate this bottleneck. By adopting a 2-way tensor parallelism and 2-way pipeline parallelism, we reduce time-to-first-token (TTFT) by 57.1% and time-per-output-token (TPOT) by 37.5%.

Finding #3. Throughput increases superlinearly when scaling pipeline parallelism on consumer-grade GPUs, but this gain diminishes under low network bandwidth and becomes negligible at 100 Mbps. We observe that throughput can increase superlinearly with the number of pipeline stages. For example, on NVIDIA RTX 2080 Ti GPUs, increasing the pipeline depth from 4 to 5 stages for Qwen3-14B at 5 Gbps network bandwidth yields a 3.3× throughput gain. This typically occurs when model weights occupy majority of GPU memory before the scaling, since the batch size is small. However, these gains diminish as network bandwidth decreases. We observe that the benefit is greatly reduced below 500mbps and finally becomes negligible at 100 Mbps, since inter-stage activation (i.e., 17KB per token) transfers become bottlenecked by limited network bandwidth.

Finding #4. State-of-the-art serving systems (e.g., vLLM and Helix) exhibit limitations when deployed on consumer-grade GPUs. *First*, They transmit outputs between pipeline stages sequentially, so decode outputs are blocked by long-running prefill outputs. As a result, GPUs remain idle waiting for prefill transmission (e.g., about 200 ms), even though decode outputs require only about 15 ms to transfer. *Second*, their deployment policies assign more layers to higher-compute GPUs to balance computational load. However, this can lead to imbalanced memory usage, particularly on consumer-grade GPUs with limited memory capacity.

To address the first issue, we propose a chunked transmission method that splits prefill outputs into smaller chunks for transmission. This enables interleaved transmission of prefill and decode data, allowing decode batches to be processed while prefill outputs are being transmitted. Our evaluation shows that this method reduces Time to First Token (TTFT) by up to 37.82%, Time Per Output Token (TPOT) by up to 18.72%, and improves overall throughput by up to 21.86% compared to the original vLLM version.

To address the second issue, we propose a bottleneck-aware deployment algorithm that leverages profiled roofline models to determine whether a node is memory-bound or compute-bound. This information guides the selection of an optimal deployment strategy to maximize overall throughput. Our evaluation shows that this approach achieves up to 11.2% higher throughput than the policy employed by Helix [6]. This improvement is especially beneficial in offline serving scenarios, where maximizing throughput is the primary objective.

To the best of our knowledge, this is the first comprehensive characterization of LLM serving from the perspectives of application, system, and hardware on consumer-grade GPUs. We hope our work lays the foundation for future research on principled co-design of models, systems, and hardware for efficient LLM serving at scale.

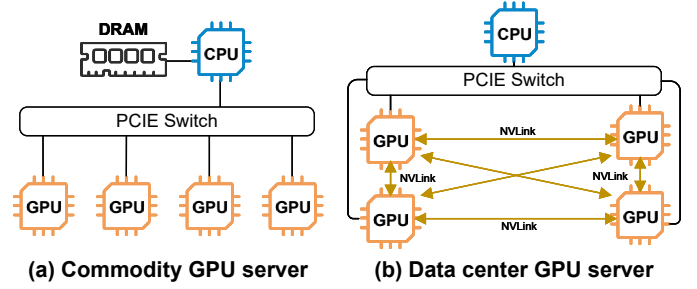


Fig. 1: The difference in architecture between commodity and data center GPU servers. In data center servers, all GPUs are additionally connected via NVLink.

II. BACKGROUND

A. LLM Inference

Recent large language models (LLMs) [13], [14] are decoder-only Transformer models trained to generate subsequent tokens for a given input sequence in an autoregressive manner. The decoder-only Transformer architecture consists of multiple decoder blocks, each comprising several components that work together to process the input data.

Model Parallelism. Since the memory of a GPU is limited, deploying an LLM often requires multiple GPUs. Two common strategies are: (1) Tensor Parallelism (TP): it distributes the weight tensor of a layer across multiple devices, either row-wise or column-wise. The devices communicate with each other to exchange activations. (2) Pipeline Parallelism (PP): the model is partitioned into consecutive layers, with each device responsible for computing its assigned segment and passing the output to the next device in the pipeline.

Prefill/Decode Inference. LLM inference typically consists of two phases: the *prefill phase* and the *decode phase*. During the prefill phase, the model processes the entire user-provided prompt to generate the first output token, which serves as the starting point for the decode phase. In contrast, the decode phase generates one token at a time, using the previously generated token as input for the next step. This process continues iteratively until a predefined maximum sequence length is reached or an end-of-sequence (EOS) token is produced.

B. Consumer-grade GPUs

Commodity GPU servers exhibit the following features.

Feature #1. The memory of a single GPU is limited. Datacenter GPUs typically offer 80 GB of memory (e.g., A100), whereas consumer-grade GPUs provide only 24 GB (e.g., RTX 4090) or less (e.g., 11 GB for RTX 2080 Ti).

Feature #2. As is shown in Figure 1, in datacenter GPU servers, GPUs are fully connected via NVLink, enabling up to 900 GB/s of bandwidth. In contrast, commodity GPU servers lack NVLink connectivity, limiting inter-GPU communication to the PCIe bus bandwidth (up to 16 GB/s).

Feature #3. In datacenter environments, LLMs are usually served on homogeneous GPUs with high-bandwidth RDMA networks. In contrast, LLM serving on consumer-grade GPUs often occurs across heterogeneous devices or even geo-distributed settings.

III. METHODOLOGY

A. Benchmarks

Recent studies evaluate various aspects of LLM serving [9]–[12]. However, they are largely limited to datacenter settings that assume homogeneous server-grade GPUs and high-bandwidth interconnects such as NVLink. In this paper, we adapt existing benchmarks and extend them to address the unique characteristics of consumer-grade hardware.

- **Model.** We use a representative open-source model family, Qwen3 [14], with parameter scales of 7B, 14B, and 32B. Detailed configurations are shown in Table I.
- **Trace.** We evaluate on three publicly available datasets: ShareGPT [15], HumenEval [16], and LongBench [17]. Dataset statistics are summarized in Table II.
- **Parallelism.** We consider both tensor parallelism and pipeline parallelism to explore their trade-offs under consumer-grade hardware constraints.

We further extend the benchmark to cover consumer-grade GPUs, heterogeneous environments, and network conditions.

- **Consumer-grade GPUs.** We consider popular consumer-grade GPUs, including the RTX 2080 Ti, RTX 3090, and RTX 4090, covering widely used models in this class.
- **Network conditions.** We evaluate a range of network configurations with bandwidths from 100 Mbps to 5 Gbps and latencies from 1 ms to 30 ms.
- **Heterogeneity.** We study heterogeneous deployments that combine RTX 2080 Ti, RTX 3090, and RTX 4090 GPUs to reflect real-world resource diversity.

To the best of our knowledge, our work is the first to provide dedicated inference benchmarking for the practical deployment of LLMs on consumer-grade GPUs.

B. Metrics

We consider the following metrics in our paper:

Time to First Token (TTFT) is the time required for the model to generate the first output token after receiving the input prompt. It reflects how quickly users see the initial response. In our measurements, TTFT is obtained by directly recording the delay between submitting the prompt and the appearance of the first token.

Time Per Output Token (TPOT) measures the average time the model spends generating each subsequent output token after the first one as shown in Eq.(1).

$$\text{TPOT} = \frac{\text{End-to-End Latency} - \text{TTFT}}{\text{Output Tokens} - 1} \quad (1)$$

Throughput is a key indicator of a hardware’s processing efficiency. In this paper, we define throughput as the number of output tokens produced by the hardware per second. We first measure the end-to-end latency, defined as the time elapsed between providing the input prompt to the LLM and generating the final output token. We then compute throughput using Eq.(2).

$$\text{Throughput} = \frac{\text{Output Tokens}}{\text{End-to-End Latency}} \quad (2)$$

Model	l	h	i	v	n_{kv}	n
Qwen3-8B	36	4096	12288	151936	8	32
Qwen3-14B	40	5120	17408	151936	8	40
Qwen3-32B	64	5120	25600	151936	8	64

TABLE I: Model hyperparameters

Dataset	Avg. Input	Avg. Output	Avg. Context
HumanEval	141.2	52.6	193.8
ShareGPT	676.1	222.2	898.3
LongBench	1666.1	87.2	1753.3

TABLE II: Length distribution of the dataset.

MFU. MFU is a crucial metric for evaluating GPU computing power utilization during LLM inference [18]. It measures the ratio between the actual computational workload incurred during inference FLOPS_f/t and the total available computational power of the GPU $\text{FLOPS}_{\text{gpus}}$. Here, t refers to the time consumed by one forward inference step.

$$\text{MFU} = \frac{\text{FLOPS}_f}{\text{FLOPS}_{\text{gpus}} \times t} \quad (3)$$

C. Framework

We evaluate the inference performance of LLMs on the following state-of-the-art (SOTA) inference frameworks.

vLLM-v1 [19]. vLLM is a state-of-the-art LLM serving system. We use vLLM-v1 (version 0.9.2), which includes the latest optimizations such as efficient GEMM kernels.

vLLM-v0 [19]. This variant uses the v0 architecture, enabled via configuration in the same codebase (version 0.9.2). Although vLLM-v0 lacks some features of vLLM-v1, we include it because vLLM-v1 does not support older consumer-grade GPUs such as the RTX 2080 Ti. When results from vLLM-v0 and vLLM-v1 conflict, we prioritize vLLM-v1 due to its more advanced feature set.

Helix [6]. Helix is the state-of-the-art system for heterogeneous environments. It abstracts each GPU node by its compute capacity and network bandwidth, formulates LLM deployment as a max-flow problem, and solves it using mixed-integer linear programming (MILP). We compare Helix with vLLM in our evaluation. Note that Helix is built on an older version of vLLM (v0.4.0).

Although SGLang [2] and Petals [4] are also representative systems, we do not include them in this study. SGLang is designed for structured LLM serving, which falls outside the scope of our focus on general-purpose inference. Petals targets decentralized, individual inference and exhibits poor throughput under standard serving workloads.

IV. MEASUREMENT

A. Model-wise Benchmarking

Experiment setup. We fix the input length to 512 tokens and the output length to 256 tokens, and generate request arrivals using a Poisson process. The model is deployed on four RTX 4090 GPUs with tensor and pipeline parallelism (TP=2, PP=2). We evaluate three models from the Qwen3 family: Qwen3-8B, Qwen3-14B, and Qwen3-32B.

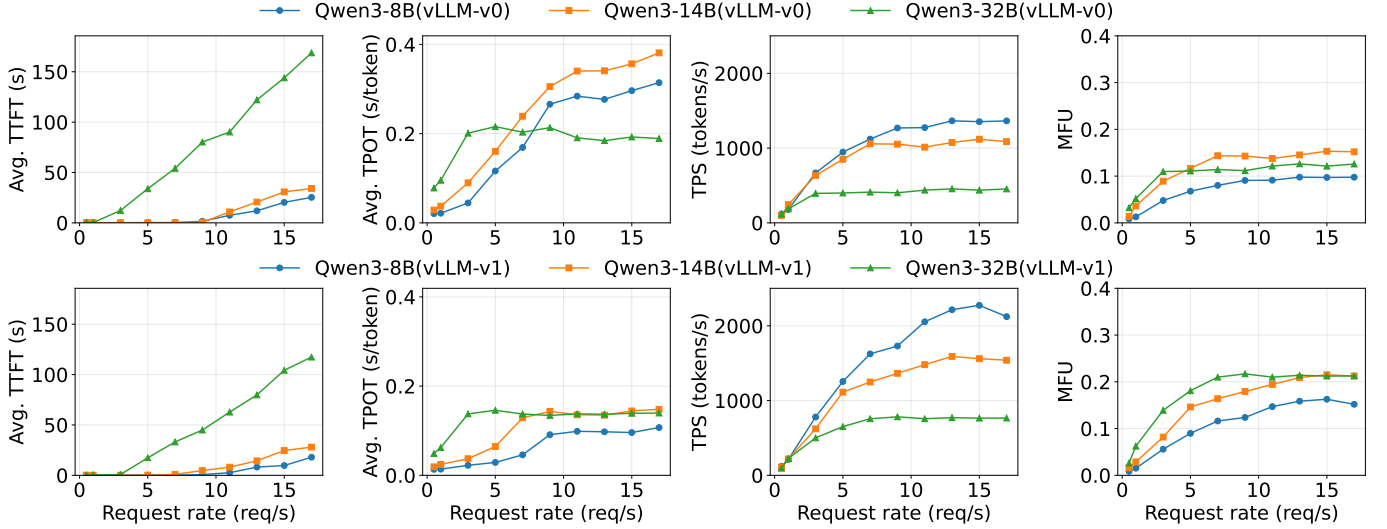


Fig. 2: Performance across model sizes. Setup: 4xRTX 4090, TP=2, PP=2, input=512, output=256.

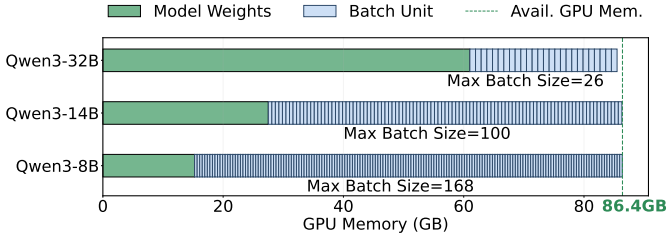


Fig. 3: GPU Memory Utilization and Maximum Batch Size in Model-wise Benchmark

Observation. The serving performance of Qwen-32B is noticeably worse than that of Qwen-8B and Qwen-14B, particularly in terms of TTFT and throughput. As shown in Figure 2, the TTFT for Qwen3-32B increases sharply with request rate, exceeding 100 seconds at 15 requests per second (req/s). In contrast, both Qwen3-8B and Qwen3-14B maintain TTFT below 50 seconds across all tested request rates. Furthermore, the throughput of Qwen3-32B remains below 1,000 tokens per second, which is more than two times lower than that of Qwen3-8B and Qwen3-14B when the request rate exceeds 5 req/s.

Observation. For TPOT, the performance of Qwen3-32B is comparable to that of Qwen3-8B and Qwen3-14B. As shown in Figure 2, the TPOT of Qwen3-32B remains consistently close to that of Qwen3-8B and Qwen3-14B across all request rates under both vLLM-v0 and vLLM-v1 setups. While Qwen3-32B shows slightly higher TPOT at low request rates, the differences diminish as the load increases, with all three models converging to similar values above 10 req/s.

Observation. The TPOT for all model sizes does not increase noticeably with request rate, especially under vLLM V1. As shown in Figure 2, the average TPOT remains nearly constant across different request rates for all models under vLLM V1, particularly for Qwen-32B and Qwen-14B, where TPOT stabilizes below 0.2 token/s even at 15 req/s. In contrast, vLLM V0 shows a slight rise in TPOT with increasing load.

Analysis. Figure 3 illustrates the components of GPU memory utilization. We assume that 90% of the GPU memory is allocated for model weights and KV cache. Under this assumption, four RTX 4090 gpus provides a total of 86.4 GB of available memory. As shown in the figure, the maximum batch size varies substantially due to significant differences in model size. For instance, the Qwen3-32B model, when loaded in half precision, consumes 61.02 GB (70.6%) of memory for weights, leaving only approximately 25 GB for the KV cache. Given a request with an input length of 512 and an output length of 256, this configuration supports about 26 concurrent requests. In contrast, the Qwen3-14B model allows up to 100 concurrent requests, while the Qwen3-8B model can support around 168 requests. These notable differences in maximum batch size explain the observed performance variations across models.

In terms of TTFT and throughput. When serving the same request rate, Qwen3-32B processes fewer concurrent requests due to its larger memory footprint. This results in more requests queuing to be scheduled, leading to higher TTFT compared to Qwen3-14B and Qwen3-8B. Additionally, each batch generates fewer tokens on average, which reduces overall throughput.

In terms of TPOT, as the request rate increases, TPOT remains stable across all model sizes and stays below 0.2 seconds. This indicates that the FLOP required by even the largest batch size, up to 168 requests, do not saturate the GPU’s computational capacity. Notably, vLLM v0 exhibits a higher TPOT as the request rate increases, likely because it uses less efficient kernels compared to vLLM v1.

Takeaway #1. Memory capacity becomes the bottleneck for serving larger models on consumer-grade GPUs. When serving a large model, its weights consume the majority of available GPU memory, which limits request concurrency. Even though these GPUs have substantial compute capability, it that remains unsaturated. Deploying smaller models can effectively improve concurrency.

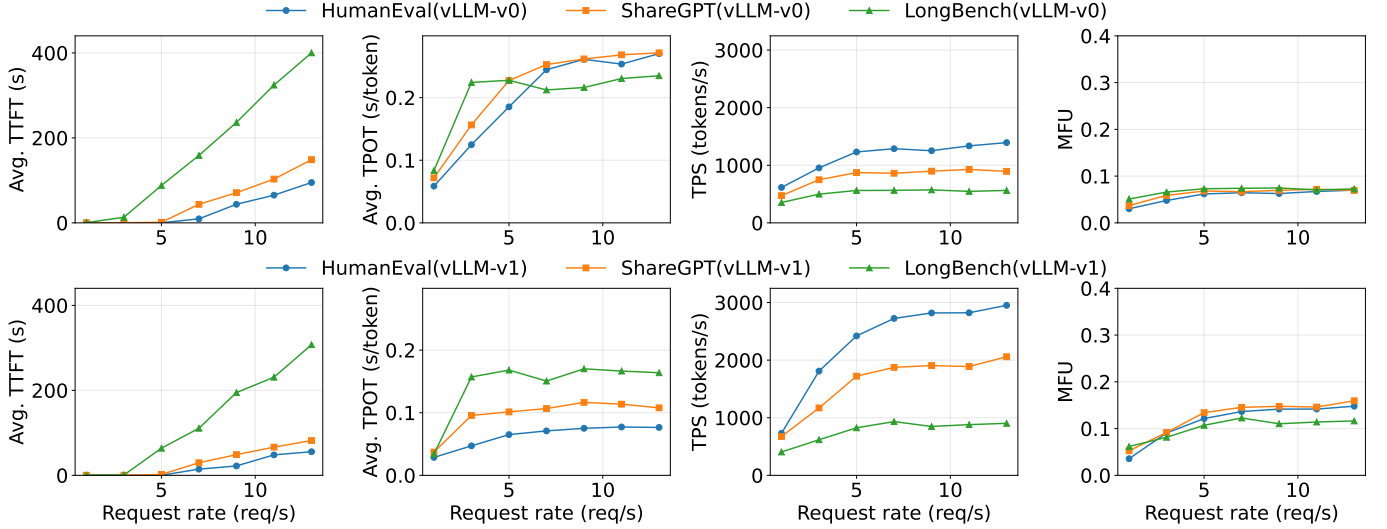


Fig. 4: Performance across datasets. Setup: 4xRTX 4090, TP=2, PP=2, model=Qwen3-14B.

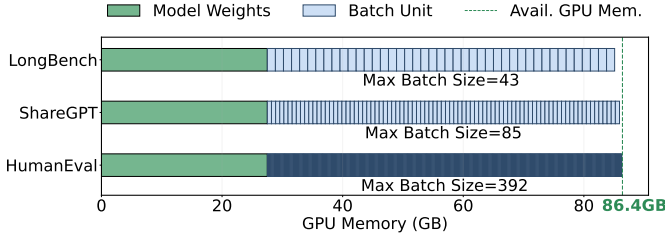


Fig. 5: GPU Memory Utilization and Maximum Batch Size in Trace-wise Benchmark

B. Trace-wise Benchmarking

Experiment setup. In this experiment, we use three datasets—HumanEval, ShareGPT, and LongBench—to construct input-output pairs, and generate request arrivals following a Poisson process with varying request rates. Their average input and output length are shown in Tab. II. The model is deployed on four RTX 4090 GPUs using tensor and pipeline parallelism (PP=2, TP=2). We use Qwen3-14B.

Observation. The serving performance of LongBench is worse than that of HumanEval and ShareGPT across all metrics. As shown in Figure 4, LongBench exhibits higher TTFT and TPOT compared to HumanEval and ShareGPT across most request rates in both vLLM-v0 and vLLM-v1 setups. Its TPS remains consistently lower, particularly under high request rates, due to reduced batch concurrency caused by long input sequences. Despite this, its Model FLOPs Utilization (MFU) stays at a similar level.

Analysis. Figure 3 illustrates the components of GPU memory utilization. The KV cache size of each request scales linearly with its sequence length, resulting in significantly smaller batch sizes for LongBench compared to ShareGPT and HumanEval. Specifically, a LongBench request with an average context length of 1,753 tokens occupies 1.6% of GPU memory, whereas a ShareGPT request (average context length: 193 tokens) uses only 0.17%. This large disparity in batch size directly degrades TTFT and throughput: fewer concurrently

served requests lead to lower token generation rates and longer queuing delays for newly arrived requests.

Takeaway #2. Memory capacity becomes the bottleneck for serving high-concurrency, long-context requests on consumer-grade GPUs. When serving long context requests, the KV cache required per request grows linearly with sequence length, quickly consuming available GPU memory. This is pronounced when context lengths exceed 1000 tokens, which is a common length in real-world chat applications.

Observation. TPOT is more sensitive to the context length than to the model size. As shown in Figure 4, the TPOT varies across datasets. LongBench exhibits higher TPOT than HumanEval and ShareGPT under vLLM-v1, especially as request rates increase. This contrasts with observation (2) in the model size benchmark, where TPOT remained similar across different model sizes (Qwen3-8B, -14B, and -32B).

Analysis. The TPOT consists of the computation time of the attention and the feed-forward network (FFN). While the FFN is compute-intensive due to large GEMM operations, the primary contributor to the increase in TPOT is the attention computation. Typically, attention is memory-bound because it loads the key-value (KV) cache from memory for each request, and the size of this cache grows linearly with the sequence length. Compared to their FLOPs capability, consumer-grade GPUs have relatively limited memory bandwidth (e.g., 1 TB/s for the RTX 4090 versus 2 TB/s for the A100). As a result, memory bandwidth becomes the bottleneck for long-context inference. Consequently, TPOT is more sensitive to context length than to model size.

Takeaway #3. Memory bandwidth becomes the bottleneck for low-latency inference when serving long-context requests on consumer-grade GPUs. This is because the attention computation is memory-bound due to frequent loading of the KV cache, and consumer-grade GPUs have relatively limited memory bandwidth. For example, 1 TB/s for the RTX 4090 compared to 2 TB/s for the A100.

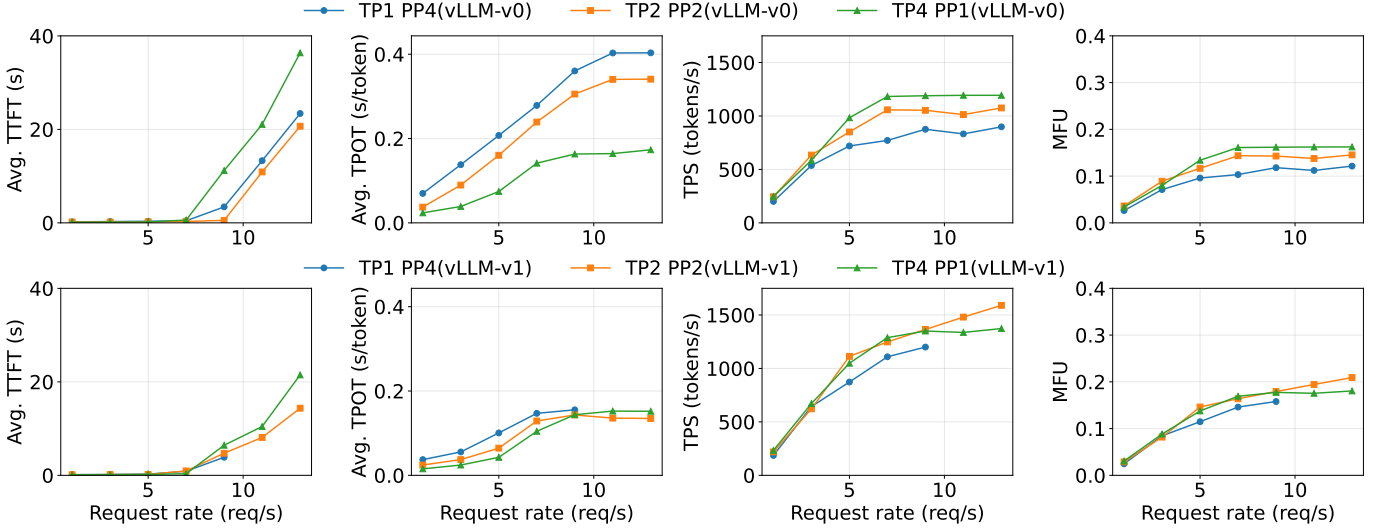


Fig. 6: Performance across different TP and PP. Setup: 4×RTX 4090, input=512, output=256, model=Qwen3-14B.

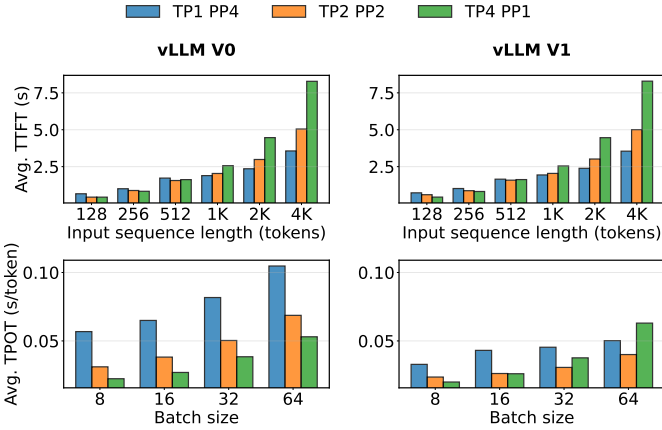


Fig. 7: Impact of sequence length and batch size on latency.

C. Parallelism-wise Benchmarking

Experiment setup. We evaluate LLM serving performance under different parallelization strategies. Input length is fixed at 512 tokens and output length at 256 tokens, with requests arriving via a Poisson process. The model (Qwen3-14B) is deployed on four RTX 4090 GPUs, and we compare three configurations: tensor-only (PP1, TP4), hybrid (PP2, TP2), and pipeline-only (PP4, TP1).

Observation. vLLM V0 and V1 exhibit different performance under pipeline parallelism, particularly in terms of TPOT. As shown in Fig. 6, vLLM-v1 significantly outperforms vLLM-v0 when pipeline parallelism is employed. For instance, under the TP2 PP2 configuration, the TPOT of vLLM-v0 is more than twice that of vLLM-v1. In contrast, their performance is comparable when using pure tensor parallelism, as both achieve similar TPOT under the TP4 PP1 configuration.

Analysis. We measure average TTFT and TPOT for a fixed input sequence length and batch size in Figure 7. We can see that the TTFT of vLLM-v0 and vLLM-v1 is almost the same. However, for TPOT, vLLM-v0 is significantly higher than vLLM-v1, especially for TP2+PP2 and TP1+PP4. We find that vLLM-v0 executes computation and communication

sequentially so that computation has to wait for transmission to finish before start. Also, we find that vLLM-v0 uses both gloo and nccl as the communication backend, which incurs an extra overhead of three times the RTT when sending data. In contrast, vLLM-v1 uses only NCCL, without extra RTT.

Takeaway #4. The efficiency of pipeline parallelism in vLLM-v0 is much lower than in vLLM-v1, while tensor parallelism performs similarly in both. This is because vLLM-v0 executes computation and communication sequentially, leading to GPU bubbles in the pipeline. In contrast, vLLM-v1 overlaps these operations, significantly reducing idle time and improving pipeline utilization.

Observation. Higher tensor parallelism (TP) does not noticeably improve latency. Instead, it may lead to worse TTFT and TPOT. As shown in Fig. 6, (TP=4, PP=1) achieves better performance when the request rate is below 9 req/s, whereas (TP=1, PP=4) performs better when the request rate exceeds 9 req/s.

Analysis. We focus on the performance metrics of vLLM-v1. Figure 7 shows that increasing the tensor parallelism (TP) degree does not improve TTFT and TPOT, especially for long input sequence lengths or large batch sizes. It is because tensor parallelism requires frequent all-reduce operation, resulting in significant inter-GPU data transfer. For example, with a sequence length of 2K on Qwen3-14B, the communication volume per attention layer for K/V AllGather is approximately 62.9 MB. At PCIe bandwidth (i.e., 32GB/s), it takes about 40 ms across all 40 layers, which is comparable to the total computation time. By adopting a hybrid parallelism strategy with 2-way tensor parallelism and 2-way pipeline parallelism, we reduce TTFT by 57.1% and TPOT by 37.5%.

Takeaway #5. Limited inter-GPU bandwidth on consumer-grade GPUs causes pure tensor parallelism to suffer from a communication bottleneck. To mitigate this, we can adopt a hybrid parallelism strategy that reduces tensor parallelism in favor of pipeline parallelism. This lowers communication volume and improves overall performance.

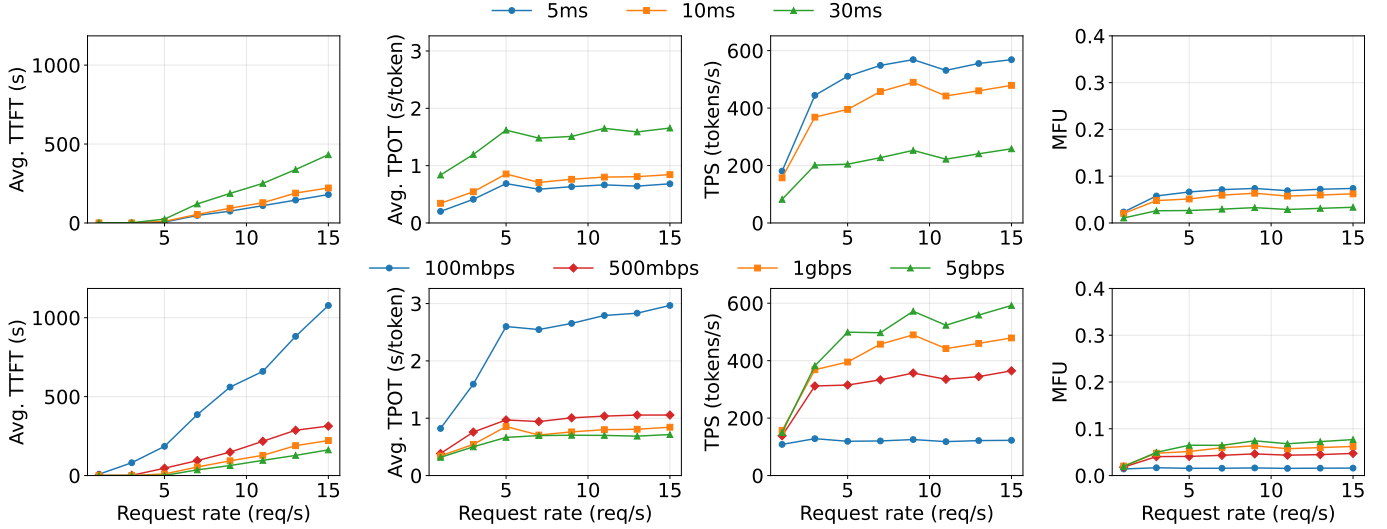


Fig. 8: Performance across network conditions. Setup: 4×RTX 4090, TP=2, PP=2, input=512, output=256.

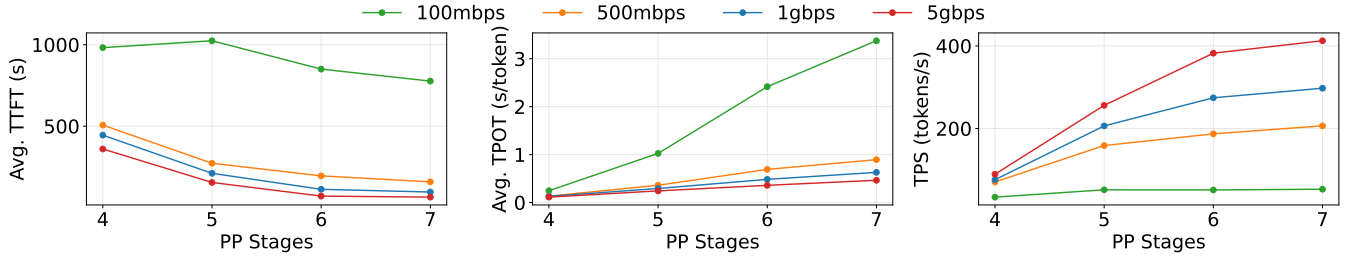


Fig. 9: Performance across pipeline stages. Setup: RTX 2080Ti. TP=1. input=512, output=256. Rate = 9 req/s.

D. Network-wise Benchmarking

Experiment setup. We fix the input size to 512 tokens and output size to 256 tokens. Request arrivals follow a Poisson distribution. We use two servers, each equipped with two RTX 4090 GPUs. We set tensor parallelism (TP) to 2 and pipeline parallelism (PP) to 2. The model is Qwen3-14B. It not specified, the bandwidth is 1Gbps and latency is 10ms. We consider different network conditions in Fig. 8. We further scale the GPUs in pipeline parallelism in Figure 9; they use 2080 Ti GPUs. We use vLLM-v0, since vLLM-v1 does not currently support 2080Ti.

Observation. Serving performance significantly degrades with increasing network latency. From Fig. 8, we observe that TTFT (Time to First Token), TPOT (Time Per Output Token), and throughput all degrade as network latency between servers increases, especially for latency of 30ms.

Analysis. Performance is noticeably impaired at latency of 30 ms. This is intuitive, as network latency accumulates across pipeline stages during each decoding iteration, slowing down the forward propagation of requests. However, the observed degradation is more severe than expected: while a 30 ms one-way latency should theoretically add only about 60 ms per decode iteration, the actual increase in latency is substantially higher. To investigate further, we find that vLLM v0 uses Gloo as its communication backend, which introduces an overhead of three round-trip times (3 RTTs) per communication step, thereby amplifying the impact of network latency.

Observation. Throughput increases superlinearly when scaling pipeline parallelism on consumer-grade GPUs, but this gain diminishes under low network bandwidth. We observe that throughput can increase superlinearly with the number of pipeline stages. For example, in Figure 9, increasing the pipeline depth from 4 to 5 stages for Qwen3-14B under a 5 Gbps network bandwidth yields a 3.3× throughput gain. However, these gains diminish as network bandwidth decreases: the speedup drops to 3.0× at 1 Gbps and further to 2.7× at 500 Mbps. When the bandwidth is reduced to 100 Mbps, it yields negligible improvement.

Analysis. The communication overhead of pipeline parallelism is small when each stage transmits the tensor only once. At 1 Gbps and 5 Gbps, the bandwidth is sufficient. For example, for Qwen-14B, whose hidden size is 17,408, a single token only needs to transfer about 17 KB. At 1 Gbps, a batch takes only about 0.1 ms to transfer, which is relatively small compared to the computation overhead—typically at least 20 ms. However, when the bandwidth drops below 100 Mbps, it takes about 1 ms per node to transfer the data. This can become larger than the computation time, especially since a batch typically consists of 10–20 tokens.

Takeaway #6. Network bandwidth must be carefully considered when scaling pipeline parallelism on consumer-grade GPUs. Memory of consumer grade gpu is limited, scaling to pipeline helps to improve throughput. However, it constraint to the bandwidth. To gain a superlinearly increase, this should at least higher than 500mbps.

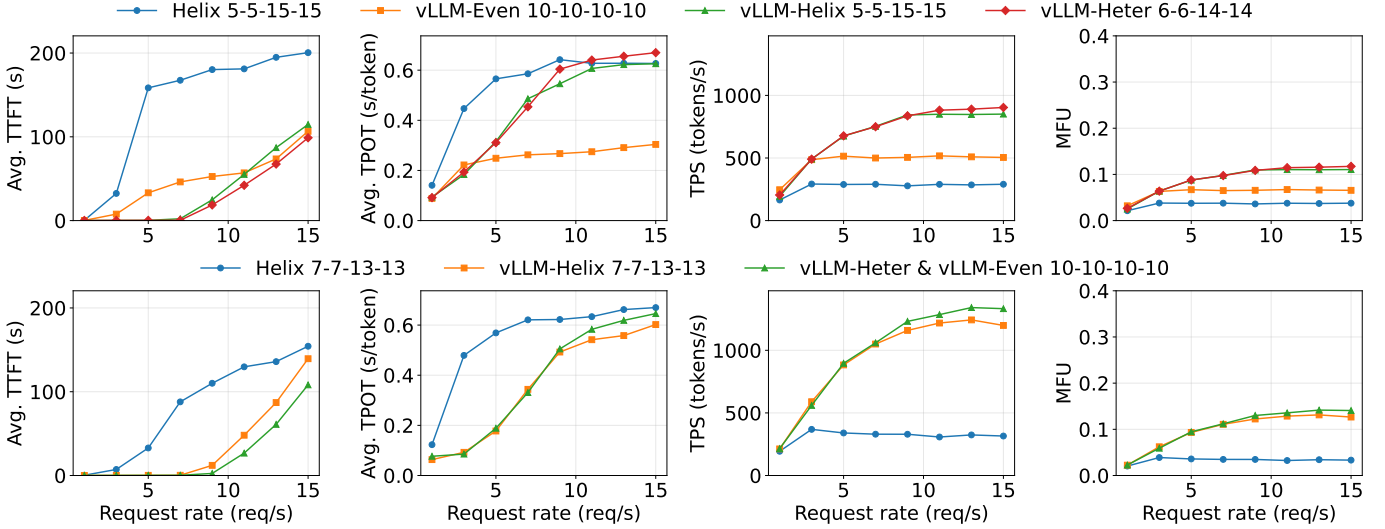


Fig. 10: Env1: 2 x 2080Ti and 2 x 4090 (the first row). Env2: 2 x 3090 and 2 x 4090 (the second row).

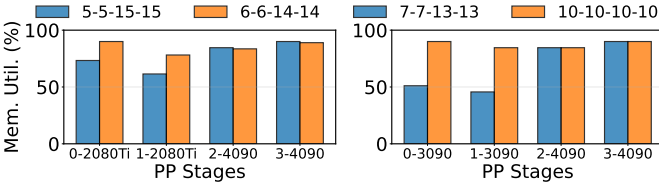


Fig. 11: The memory utilization of each PP stages in Env1 and Env2. The request rate is 9 req/s.

E. Heterogeneity-wise Benchmarking

Experiment setup. We use LongBench as the evaluation dataset and model request arrivals using a Poisson distribution. We employ two heterogeneous clusters: Env1 consists of two RTX 2080 Ti and two RTX 4090 GPUs, while Env2 consists of two RTX 3090 and two RTX 4090 GPUs. The intra-cluster network provides 10 Gbps bandwidth with average latency below 1 ms, whereas the inter-cluster network offers 1 Gbps bandwidth with an average latency of 10 ms. In this experiment, we focus on model splitting and therefore use pipeline parallelism with a degree of 4.

Baselines. (1) vLLM-Even, which evenly splits the model across GPUs. (2) Helix [6], the state-of-the-art LLM serving system for heterogeneous clusters. (3) vLLM-Helix, which follows Helix’s splitting strategy. Both vLLM variants use v0.9.2 with architecture V0, as architecture V1 does not support the RTX 2080Ti GPU. We evaluate their serving performance and memory utilization in Figure 10.

Observation. vLLM-Helix splits the model slice according to compute capacity and achieves better performance in Env1. From Figure 10, vLLM-Helix assigns model layers proportionally to GPU TFLOPs. For example, it allocates 5 layers to the 2080Ti and 15 to the 4090, closely matching their compute capacity ratio. Helix uses compute capacity as an abstraction of GPU capability, assigning more layers to higher-capacity GPUs like the 4090. These profiles are collected offline once during setup. In contrast, vLLM-Even assigns 10 layers to each GPU regardless of hardware differences,

resulting in 79% lower throughput than vLLM-Helix.

Observation. vLLM-Helix performs worse than vLLM-Even in Env2 due to memory under-utilization. As shown in Figure 10, vLLM-Helix achieves lower throughput than vLLM-Even. The cause is shown in Figure 11. vLLM-Helix assigns an average of 7 model layers to the RTX 3090 and 13 to the RTX 4090 based on their compute capacities. However, this uneven partitioning leads to imbalanced memory usage for both model weights and KV cache. For instance, the RTX 4090 consumes nearly twice as much memory as the RTX 3090. Consequently, when the RTX 4090 reaches full memory utilization, the RTX 3090 uses only about 50 of its capacity, causing significant under-utilization. As a result, vLLM-Even achieves 11% higher throughput than vLLM-Helix.

An Optimization Opportunity: Bottleneck-Aware Deployment. Helix tends to assign more model weight to GPUs with higher compute capacity. However, on consumer-grade GPUs, compute capacity is often underutilized due to memory constraints, which limit the number of concurrent requests. This can lead to imbalanced memory usage and reduced overall throughput. To address this issue, we propose an algorithm based on a profiled roofline model to determine the optimal model deployment. This model identifies whether a node is memory-bound or compute-bound, enabling deployment decisions that better match hardware characteristics.

Cost Model. The cost model predict the performance of the distributed gpus when serving LLM. First, we follow the original definition of the roofline model [20]. We set P_{peak} according to a profiling result, since it is correlated with the implementation of the inference system, such as the GEMM kernel used. This also aligns with Helix [6], which uses profiling results as the peak compute capability ($P_{\text{peak}} = P_{\text{profiled}}$). Therefore, for a GPU, we have:

$$P = \min(P_{\text{peak}}, I * B) \quad (4)$$

where (1) P_{peak} is the peak computational capacity of the given hardware, expressed in FLOPs/s; (2) B is the memory

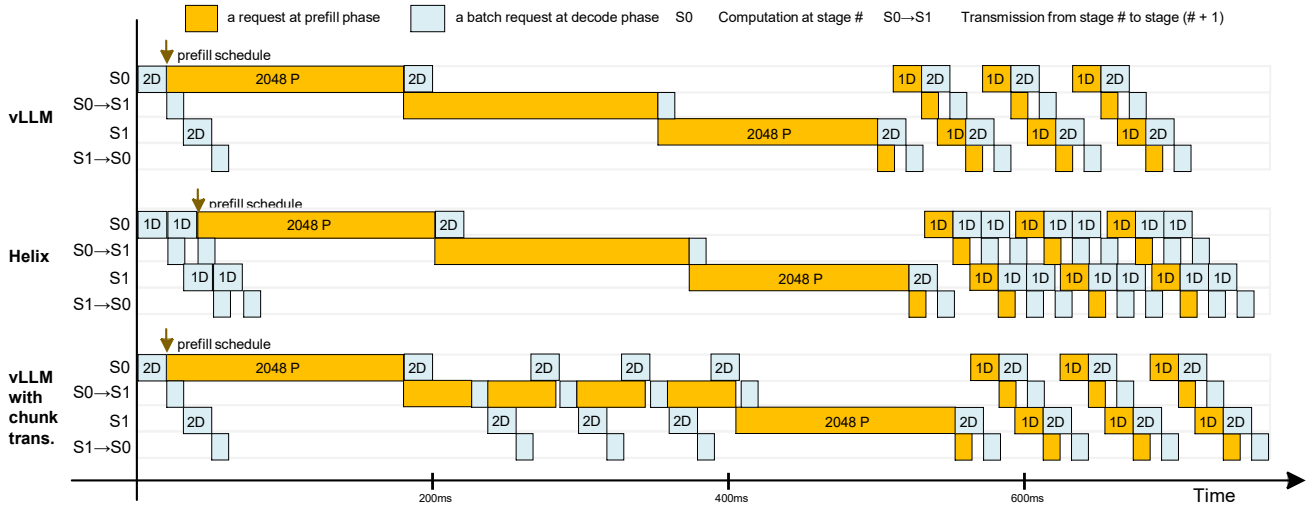


Fig. 12: The batch and data scheduling of serving systems. The number indicates the number of tokens processed in the batch. “P” denotes prefill while “D” denotes decode. “S0” denotes the compute node. “S0→S1” denotes the transmission process.

bandwidth of the given hardware, expressed in Bytes/s; and (3) I is the operational intensity, expressed in FLOPs/Byte.

The ratio of the number of floating-point operations required to process one token per unit weight is a constant, denoted as i_{per_token} . Since a unit of weight can be reused across a batch of tokens (bz), the operational intensity (I) for per unit weight scales linearly with the batch size.

$$I = i_{per_token} * bz \quad (5)$$

When forwarding through the model layers to generate a token, the FLOPs are accumulated per unit of *weight*. Therefore, the total FLOPs required to generate one token ($flops_per_token$) is proportional to the *weight*.

$$flops_per_token = i_{per_token} * weight \quad (6)$$

The token throughput (tp) of this GPU can be calculated as the ratio of its peak FLOPs rate to the FLOPs required per token. Therefore, we have:

$$tp = P / flops_per_token \quad (7)$$

The total number of parameters is W . When a set of GPUs $\{gpu_i\}$ is given, we denote the assigned weight and throughput for gpu_i as $weight_i$ and tp_i , respectively. The batch size bz is same. We denote M_i as its memory capacity. Its peak memory usage ($peak_mem_i$) increases with $weight_i$ and bz , since a larger batch size results in a larger KV cache. Our goal is to:

$$\begin{aligned} & \underset{\{weight_i, bz\}}{\text{minimize}} && 1/Throughput \\ & \text{subject to} && \sum weight_i = W, \\ & && Throughput \leq tp_i, \quad i = 1, 2, \dots, n, \\ & && peak_mem_i < M_i, \quad i = 1, 2, \dots, n. \end{aligned} \quad (8)$$

Policy Search. This model is simple, with variables consisting of a set $\{weight_i\}$ and bz . We enumerate all feasible configurations and select the one that yields the highest throughput. The computational complexity increases with the number of GPUs. This can be addressed via a two-level optimization: first enumerate and fix b , then reduce the problem to a mixed-

integer linear program (MILP) over the remaining variables. A similar approach is used in [21].

Evaluation. Figure 10 presents the evaluation results of our method. Our approach is vLLM-Heter. We observe that our approach improves overall throughput under high workloads, specifically when the request rate exceeds 10 requests per second (req/s). In Env1, our method achieves a 6.2% higher throughput compared to the baseline, and in Env2, the improvement reaches 11.2%. This is especially beneficial for offline serving scenarios, where the primary objective is maximizing throughput. This setting is also the target of Helix.

F. Pipeline Scheduling Analysis

Experiment setup. We use LongBench as dataset. Model request arrivals with the Poisson distribution. We record timestamps marking the start and end of each batch. We virtualize the computation and communication pipeline of two state-of-the-art systems, Helix and vLLM, under a network configuration with 1 Gbps bandwidth and 10 ms latency, which represents a typical setting for standard Ethernet without RDMA. Our experimental setup employs two RTX 4090 GPUs with pipeline parallelism and the Qwen-14B.

Observation. Both prefill and decode transmissions incur non-negligible overhead, leading to GPU idle time. As shown in Figure 12, the transmission overhead for both prefill and decode phases is significant. While GPUs wait for the transmission of prefill or decode outputs, they remain idle.

In the timeline from 0 ms to 500 ms, a prefill request with a sequence length of 2048 takes approximately 200 ms to transmit, of which about 180 ms is spent sending data over the network due to bandwidth constraints. During this period, both GPUs in vLLM and Helix remain idle for roughly 200 ms while waiting for the prefill output to arrive.

In the timeline from 500 ms to 700 ms, decode outputs exhibit similar behavior: under the given network latency and bandwidth, each transmission typically takes at least 10 ms to complete. Although intuition suggests that overlapping two batches should nearly eliminate pipeline bubbles [22], this

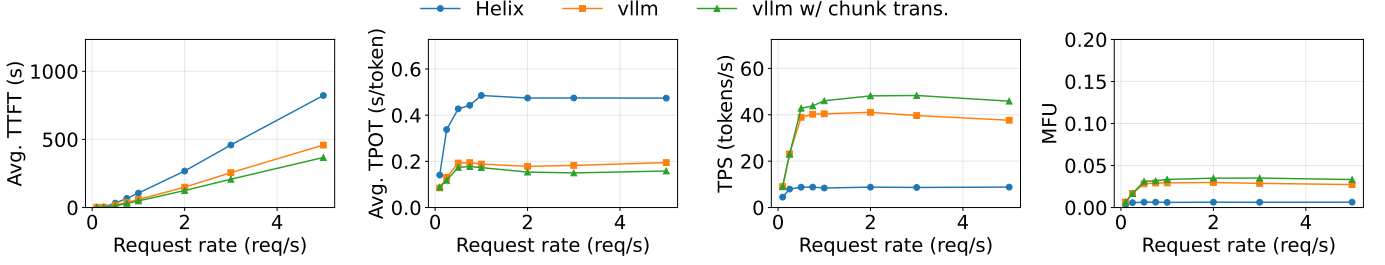


Fig. 13: The performance of serving systems. Setup: 2xRTX 4090, PP=2, input=512, output=256.

does not hold under such network-constrained conditions. In vLLM, which uses two micro-batches executed in parallel in the pipeline, bubbles still occur. Helix increases the number of concurrent batches to three, which helps mitigate the issue by filling the idle periods with additional batch computation.

An Optimization Opportunity: Chunked transmission.

As shown in Figure 12, the prefill stage takes significantly longer than the decode stage, which can block the transmission of decode batches and lead to GPU idle time. To address this issue, we propose a chunked transmission method that splits prefill data into smaller chunks. This enables interleaved transmission of prefill and decode data, allowing decode batches to be executed while prefill outputs are being transmitted.

Algorithm 1 Transmission schedule

```

1: Initialize data queue  $q_1, q_2 \leftarrow \emptyset$ 
2: While True do
3:    $v = \text{get\_decode\_volume}()$ 
4:   if  $v \neq \text{None}$ : add  $v$  to  $q_1$ 
5:    $v = \text{get\_prefill\_volume}()$ 
6:   if  $v \neq \text{None}$ : split  $v$  into  $\{v_1, \dots, v_n\}$  and add it to  $q_2$ 
7:   if  $q_1 \neq \emptyset$ : send(pop( $q_1$ ))
8:   elif  $q_2 \neq \emptyset$ : send(pop( $q_2$ ))

```

Algorithm 1 shows our schedule algorithm of transmission. The decode volume is added into queue q_1 , while the prefill volume is split into several chunks $\{v_1, \dots, v_n\}$ before added into queue q_2 (line 1-6). We priorities decode requests. Therefore, we choose to transmit a decode request even there exists prefill requests needing transmission (line 7-8). The chunk size is determined by both computation time and transmission time, as illustrated in Figure 12. Specifically, the optimal chunk size corresponds to two computation intervals plus one transmission interval. In this case, we profile a typical batch size of 20, which incurs approximately 20 ms of computation time and 12 ms of transmission time. The chunk size can be calculated as: $(2 * 20\text{ms} + 10\text{ms}) * 1\text{gbps} = 6.25\text{MB}$.

As shown in Figure 12, our method (i.e., vLLM with chunk trans.) enables the pipeline to complete three additional decode iterations without significantly delaying the prefill phase. Note that this chunk is transmitted during a single iteration. Our approach differs from chunked prefill [22], which determines chunk size based solely on computational capacity. In contrast, our method adapts the chunk size according to both hardware characteristics and network bandwidth.

Evaluation. Figure 13 presents the performance results. Compared to vLLM, our method reduces time-to-first-token

(TTFT) by up to 37.82%, decreases time-per-output-token (TPOT) by up to 18.72%, and improves overall throughput by up to 21.86%. It is worth noting that while chunked transmission may slightly prolong the total prefill transmission time, the reduction in queuing delay more than compensates for this overhead, resulting in a net decrease in TTFT.

V. RELATED WORK

LLM Inference Benchmarks. Many studies have focused on benchmarking and characterizing deep learning workloads [23]–[27]. These works have been evaluating using traditional deep learning benchmarks. Recently, several system-level studies have specifically investigated LLM inference performance [9]–[12]. For instance, qMeter [11] analyzes quantization methods for LLMs. Our work is complementary: we focus on standard floating-point precision (e.g., FP16) rather than quantized representations. CPU-Bench [12] examines LLM inference on CPUs, which is orthogonal to our work. The most closely related work is LLM-Inference-Bench [9], which evaluates LLM inference across accelerator platforms. However, it targets server-grade hardware such as the NVIDIA H100. These devices typically offer significantly higher memory capacity and bandwidth compared to consumer-grade GPUs.

LLM Inference Optimizations. Many LLM serving systems have proposed optimizations for consumer-grade GPUs [2]–[8]. A representative technique is PagedAttention, introduced in vLLM [5], which allocates a paged KV cache; this approach is orthogonal to our work. Other works target heterogeneous consumer GPU environments. For instance, Helix [6] finds the optimal deployment strategy to maximize LLM serving throughput in such settings, and it serves as a baseline for our work. Another related approach is chunked prefill [22], which determines chunk size based solely on computational capacity. Our work is complementary, as we operate at the transmission level. For example, when the original chunk size is 512 tokens, we may further split it into 256-token chunks during transmission if the available bandwidth is 500 Mbps. Our approach adaptively adjusts the transmission chunk size based on both hardware characteristics and network bandwidth.

VI. CONCLUSION

This paper presents the first comprehensive characterization of LLM serving from the perspectives of application, system, and hardware on consumer-grade GPUs. We hope our work lays foundation for future research on efficient LLM serving.

REFERENCES

- [1] jonpeddie. Q4'21 sees a nominal rise in GPU and PC shipments quarter-to-quarter. <https://www.jonpeddie.com/news/q421-sees-a-nominal-rise-in-gpu-and-pc-shipments-quarter-to-quarter/>, 2024.
- [2] Lianmin Zheng, Liangsheng Yin, and et al. SGLang: Efficient Execution of Structured Language Model Programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [3] Youhe Jiang and et al. HexGen: Generative Inference of Large Language Model over Heterogeneous Environment. In *Proceedings of the 41st International Conference on Machine Learning*, volume 16, 2024.
- [4] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Ribabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative Inference and Fine-Tuning of Large Models. In *The 61st Annual Meeting of the Association for Computational Linguistics (Volume 3)*, pages 558–568, 2023.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving With Pagedattention. In *The 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [6] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, and et al. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 586–602, 2025.
- [7] Kexin Chu, Zixu Shen, Sheng-Ru Cheng, Dawei Xiang, Ziqin Liu, and Wei Zhang. Mcam: Efficient LLM Inference With Multi-Tier Kv Cache Management. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*, pages 571–581, 2025.
- [8] JinYi Yoon, JiHo Lee, Ting He, Nakjung Choi, and Bo Ji. S2M3: Split-And-Share Multi-Modal Models for Distributed Multi-Task Inference on the Edge. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*, pages 868–878, 2025.
- [9] Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffanetti, Valerie Taylor, Murali Emani, and Venkatram Vishwanath. LLM-Inference-Bench: Inference Benchmarking of Large Language Models on AI Accelerators. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1362–1379, 2024.
- [10] Konstantinos Papaioannou and et al. The Importance of Workload Choice in Evaluating LLM Inference Systems. In *4th Workshop on Machine Learning and Systems*, pages 39–46, 2024.
- [11] Tianyao Shi and Yi Ding. Systematic Characterization of LLM Quantization: A Performance, Energy, and Quality Perspective. *arXiv preprint arXiv:2508.16712*, 2025.
- [12] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Jeffrey Young, Tushar Krishna, and Hyesoon Kim. Understanding Performance Implications of LLM Inference on CPUs. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*, pages 169–180, 2024.
- [13] OpenAI. GPT-4 Technical Report, 2023.
- [14] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
- [15] ShareAI Lab. ShareGPT-Chinese-English-90k: A Bilingual Chinese-English Human-Machine Dialogue Dataset. <https://huggingface.co/datasets/shareAI/ShareGPT-Chinese-English-90k>, 2023. Hugging Face dataset repository.
- [16] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, and et al. Evaluating Large Language Models Trained on Code. 2021.
- [17] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A Bilingual, multitask benchmark for long context understanding, 2023.
- [18] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, and et al. PaLM: Scaling Language Modeling With Pathways. *J. Mach. Learn. Res.*, 24(1), January 2023.
- [19] vllm project. vLLM. <https://github.com/vllm-project/vllm>, 2025.
- [20] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [21] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-Throughput Generative Inference of Large Language Models With a Single GPU. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.
- [22] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference With Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134, 2024.
- [23] Junqi Yin, Sajal Dash, and et al. Evaluation of Pre-Training Large Language Models on Leadership-Class Supercomputers. *The Journal of Supercomputing*, 79(18):20747–20768, 2023.
- [24] Murali Emani, Sam Foreman, Varuni Sastry, Zhen Xie, Siddhisanket Raskar, William Arnold, Rajeev Thakur, Venkatram Vishwanath, Michael E Papka, Sanjif Shanmugavelu, et al. Toward a Holistic Performance Evaluation of Large Language Models Across Diverse AI Accelerators. In *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 1–10, 2024.
- [25] Murali Emani, Zhen Xie, Siddhisanket Raskar, and et al. A Comprehensive Evaluation of Novel AI Accelerators for Deep Learning Workloads. In *2022 IEEE/ACM International Workshop on Performance Modeling, benchmarking and simulation of high performance computer systems (PMBS)*, pages 13–25, 2022.
- [26] Junqi Yin, Aristeidis Tsaris, Sajal Dash, Ross Miller, Feiyi Wang, and Mallikarjun Arjun Shankar. Comparative Evaluation of Deep Learning Workloads for Leadership-Class Systems. *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, 1(1):100005, 2021.
- [27] Murali Emani, Sam Foreman, Varuni Sastry, Zhen Xie, Siddhisanket Raskar, and et al. Toward a Holistic Performance Evaluation of Large Language Models Across Diverse AI Accelerators. In *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 1–10, 2024.